

Load Balancing

 systemdesignschool.io/fundamentals/system-design-components



System Design Components

System Design Components

Load balancing distributes network or application traffic across a number of servers. This can improve availability by ensuring no single server becomes a bottleneck or point of failure. It also aids scalability by allowing systems to handle increased traffic by distributing the load. Furthermore, load balancing can improve latency by reducing the time it takes for a server to respond to a request because the load is eventually distributed. How This Affects Non-functional Requirements:

- **Availability:** Load balancing improves availability by distributing network traffic across multiple servers, eliminating single points of failure. If one server goes down, the load balancer redirects traffic to the remaining operational servers.
- **Scalability:** By evenly distributing load, load balancers allow more requests to be served simultaneously, supporting system scaling.
- **Latency:** Load balancing can reduce latency by ensuring that no individual server is overwhelmed with traffic, maintaining quick response times.

Caching

Caching is the process of storing a copy of data in a temporary storage area (cache) so future requests for that data are served up faster. Caching contributes to scalability by reducing the load on the database or primary data source, thus allowing the system to serve more users. It

can also improve latency by reducing the time it takes to fetch data. However, caching could pose challenges to maintaining consistency, especially in a distributed system, if not properly managed.

How This Affects Non-functional Requirements:

- **Availability:** Caching can indirectly enhance availability by reducing the load on the system, which can help prevent system overloads or crashes.
- **Scalability:** By storing frequently accessed data and serving it quickly, caching reduces load on the primary data source, which supports system scaling.
- **Latency:** By serving stored data much more quickly than the primary data source could, caching significantly reduces data retrieval times, thereby reducing latency.
- **Consistency:** Caching can lead to the issue of stale data. This occurs when the original data in the database is updated, but the cached version remains unchanged. As a result, users may receive outdated information, creating a discrepancy between what is stored in the cache and the current data in the database, leading to inconsistencies.

Partitioning

Partitioning (or sharding) is the process of dividing a database into smaller, more manageable pieces. It can significantly boost scalability by allowing a system to store and process more data than a single DBMS could handle. Partitioning can also help with latency by reducing the time it takes to query large databases. However, it can create challenges for consistency if different parts of the data need to be kept in sync across partitions.

How This Affects Non-functional Requirements:

- **Availability:** Partitioning improves availability by ensuring that even if a subset of the data is inaccessible due to failures in one partition, other partitions can still serve their data. In addition, if partitioning is combined with replication, even if one partition fails, a replica can continue to serve the data.
- **Scalability:** Partitioning enhances scalability as the data is divided among multiple nodes or servers, allowing the system to handle more requests concurrently. As the system grows, new partitions can be added to distribute the data further.
- **Latency:** By keeping related data in the same partition, partitioning can help reduce latency. This is because queries can be routed to the specific partition where the data resides, avoiding the need to search the entire dataset.
- **Throughput:** Partitioning improves throughput by allowing for parallel processing. Since data is distributed across multiple partitions, different queries can be executed on different partitions concurrently. This increases the total number of operations the system can handle per second.
- **Consistency:** Partitioning can make consistency more challenging to maintain, especially in a distributed system where data is partitioned across multiple nodes. However, with proper design and the use of protocols like two-phase commit, partitioning can be compatible with strong consistency. For example, data that needs to be consistently read and written can be kept in the same partition.

Replication

Replication involves creating and maintaining multiple copies of data. It contributes to availability by providing backup data sources if the primary source fails. Replication also aids scalability by allowing read requests to be distributed across multiple copies, reducing the load on any one server. However, like caching and partitioning, replication can pose challenges for consistency, especially in cases of write operations.

How This Affects Non-functional Requirements:

- **Availability:** By creating backup copies of data, replication enhances availability. If the primary data source fails, the system can continue operation using the backup data sources.
- **Scalability:** By allowing read requests to be distributed across multiple copies of the data, replication can reduce the load on any one server, enhancing scalability.
- **Consistency:** When we have several copies of the same data, we need to make sure they all change together. But this can be tricky. If one copy gets updated and the others don't, then people might see different things when they look at the data. We use different rules, like strong consistency, eventual consistency, and causal consistency, to help make sure that all the copies stay the same, or at least get updated eventually.

Message Queue

A message queue provides an asynchronous communications protocol, meaning that the sender and receiver of the message do not need to interact with the message queue at the same time. This can improve scalability by offloading tasks from the main application threads and handling them asynchronously. Message queues can help with availability and reliability, as they often persist messages until they are processed, ensuring that important tasks are not lost even if a component fails.

How This Affects Non-functional Requirements:

- **Availability:** Message queues often persist messages until they are processed, which means that important tasks are not lost even if a component fails, enhancing availability.
- **Scalability:** Message queues can offload tasks from the main application threads and handle them asynchronously, which can help in managing larger volumes of tasks, thus supporting system scaling.

Batch Processing

Batch processing refers to the execution of a series of jobs all at once instead of individually. It improves throughput by allowing you to process large volumes of data at once, usually during off-peak times. This can be particularly useful for non-interactive jobs like data analysis or backups, where the time taken to complete the task is less critical. However, batch processing could increase latency for individual tasks within the batch, as they have to wait their turn for processing.

How This Affects Non-functional Requirements:

Scalability: By grouping similar tasks together and processing them as a unit, batch processing can handle large volumes of data more efficiently than processing each task individually, enhancing scalability.